

Sentinel — High-Level Design

Statewide CCTV Integration Platform • Model 1 + Model 2 Gujarat CCTV Integration Hackathon 2026 • Category 1 (Academic / Research / Startup)

Document	Technical Proposal / High-Level Design
Scope	Model 1 — Central CCTV Registry & GIS Mapping (compulsory) Model 2 — Unified Viewing Platform with ANPR & Watchlist Alerting
Prototype status	Running against the live Sentinel sandbox grid (30 cameras)
Companion	<i>Statewide CCTV Integration Programme — Technical Solution Document v1.0</i> ("the Tech Doc")
Measurements	All figures quoted here are reproduced in <code>DOCS/MEASUREMENTS.md</code>

1. What we built, and what we did not

Sentinel is a working platform, not a mock-up. It onboards cameras from the sandbox catalogue, plots them on a GIS map, streams them in a unified viewer, continuously reads number plates from live feeds, matches them against a watchlist, raises alerts in real time, reconstructs a vehicle's route across cameras, and exports the output report.

We are equally explicit about what is **described rather than demonstrated**. Section 9 sets out the scale-out path to 80,000 cameras; that section is a design argument supported by arithmetic, not a claim about running code. The distinction matters more than any individual feature, because a submission that blurs it cannot be trusted on the parts that *are* real.

1.1 Demonstrated in the prototype

- Camera registry populated from the live sandbox catalogue, with location provenance recorded per camera
- GIS map with department, status and location-confidence layers
- Multi-camera live viewing (HLS grid, WebRTC hero tile)
- Continuous ANPR pipeline on a subset of cameras — tiled inference, track-level voting and Indian plate-grammar correction (yield on this grid: see §6.5)
- Detection index searchable by exact, partial and trigram-fuzzy plate
- Watchlist with bulk import; automatic exact-then-fuzzy matching
- Real-time alerts pushed over WebSocket
- Route reconstruction with speed computation and impossible-transition flagging
- Output report in XLSX and PDF
- Audit log on every search, route reconstruction and export
- Role model: state admin / department operator / viewer

1.2 Described, not built

Kafka, Elasticsearch, Kubernetes, federation middleware (Model 3), face recognition, crowd analytics, vehicle re-identification, and live VAHAN / eGujCop integration. Each is addressed in Section 9 or Section 10 as part of the scale-out or integration path. At 30–50 cameras PostgreSQL does all of it, and adding those components would have cost the prototype without improving the demonstration.

1.3 Why Models 1 and 2, and not 3 or 4

We chose Model 1 + Model 2 deliberately, not by default, and the reasoning is worth stating because the four models are not a menu to be collected.

Models 2, 3 and 4 are competing answers to the same question. All three address departmental fragmentation, and they differ in where the integration sits. Model 2 connects directly to departmental endpoints and optimises for the operator's experience. Model 3 inserts a federation layer and optimises for systemic interoperability. Model 4 replaces both with a consolidated central VMS. Submitting several of them would not demonstrate more capability; it would demonstrate that we had not decided which architecture we were arguing for.

Model 1 is the exception, because it is foundational rather than alternative: a registry of what cameras exist and where they are is a precondition for every other model. So Model 1 pairs with exactly one of 2, 3 or 4, and the pairing is the choice.

Why Model 2 is the right pairing at this scale. The sandbox is thirty cameras under one operator. Federation middleware solves a problem that appears when several departments each run their own VMS with their own vendor, and none of that is present here — a federation layer over a single grid would be ceremony around a problem we do not have. Model 2 gets the operator to a working unified view, which is what a thirty-camera deployment actually needs.

What we would need to build Model 3 honestly. Its deliverable is a middleware federating *at least two different* VMS platforms. We have access to one. Federating two adapters we wrote ourselves would prove nothing, because the entire difficulty of federation is that other vendors' systems are heterogeneous, inconsistently documented and behave in ways the specification does not predict. A demonstration against two mocks of our own design tests our understanding of our own mocks.

What we would need to build Model 4 honestly. Its deliverables include a load-test report at approximately 80,000 cameras, GPU-based inference, distributed object storage, orchestration and disaster recovery. Our stated constraint is a single CPU machine with no GPU. We could draw that architecture, and Section 9 does argue the scale-out path with arithmetic — but we could not produce the load-test report, and a scalability claim without a measurement is the kind of assertion this document is organised to avoid.

Model 4 also requires face recognition, which is the highest-risk capability in the programme. Building it as a hackathon feature, against government CCTV, on a timescale that permits no proportionality review, is not a decision we were willing to take lightly.

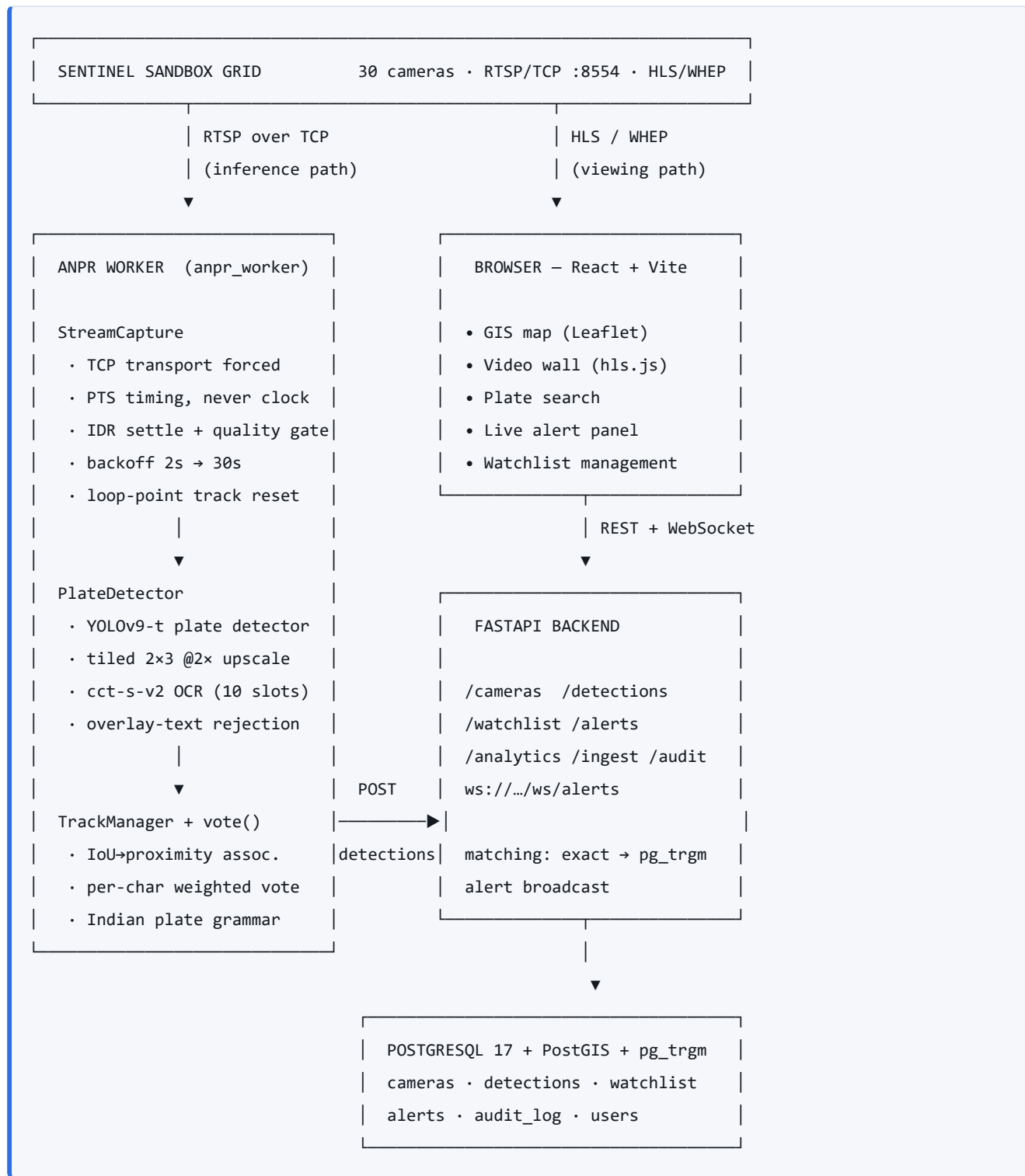
What this decision cost us, stated plainly. Two models' worth of scope. What it bought is that every claim in this submission is either running code or an argument explicitly labelled as an argument. Section 6.5 reports a measured zero-plate yield across the sandbox grid rather than a demonstration arranged to succeed; that finding exists because the effort went into measuring one thing properly instead of building four things partially.

When each of the others becomes the right choice. Model 3 earns its keep at the point where a second department with an incompatible VMS is onboarded and direct connection stops scaling — the

adapter work Section 3 describes per vendor is precisely what a federation layer exists to centralise. Model 4 follows when central retention and statewide analytics justify the storage, GPU and orchestration it requires; Section 9.3 gives the camera-count thresholds at which the components deferred in §1.2 stop being optional.

2. Architecture

2.1 Prototype topology



Every frame is decoded and analysed inside the deployment. No video, and no crop of a frame, is sent to any external service. The only outbound calls the platform makes are to OpenStreetMap Nominatim (one-off camera geocoding, cached to disk) and the OSRM demo server (optional road-snapping of a drawn route) — neither is in the detection or alerting path, and both degrade gracefully when unavailable.

2.2 Component responsibilities

Component	Responsibility	Key module
Sandbox client	Authenticated catalogue fetch; commits <code>catalogue.json</code>	<code>app/sandbox_client.py</code>
Ingest	Camera onboarding, location resolution, provenance	<code>app/routers/ingest.py</code> , <code>app/geocoding.py</code>
Capture	RTSP/TCP, PTS timing, reconnect, quality gate, loop detection	<code>app/vision.py</code>
Detection	Tiled inference, dedup, overlay rejection	<code>app/vision.py</code>
Aggregation	Track association, per-character weighted voting	<code>app/vision.py</code> , <code>app/plate_grammar.py</code>
Correlation	Exact then fuzzy watchlist matching; alert creation	<code>app/routers/watchlist.py</code>
Distribution	WebSocket alert broadcast	<code>app/websocket_manager.py</code>
Analytics	Search, route reconstruction, reports	<code>app/routers/detections.py</code> , <code>app/reporting.py</code>
Accountability	Audit trail with purpose and case reference	<code>app/audit.py</code>

3. Integrating heterogeneous cameras, NVRs and VMS

The sandbox presents one protocol family, but a statewide platform will not. The design point is that **nothing above the connector layer knows what a camera is**.

A camera enters the registry as a record with a stream URL and a codec hint. The capture layer takes a URL and yields frames. Everything downstream — indexing, matching, mapping, reporting — operates on `Detection` and `Camera` rows and is therefore already protocol-agnostic.

Source class	Integration route	Prototype status
ONVIF Profile S/T cameras	ONVIF device discovery → RTSP URI	Design
Direct RTSP / RTSPS	Native; the sandbox path	Working
HLS / WebRTC (WHEP) gateways	Native; the viewing path	Working
NVR/DVR with RTSP export	Per-channel RTSP URLs registered as cameras	Design
Proprietary VMS (Milestone, Genetec, Hikvision)	Vendor SDK/API adapter behind the same connector interface	Design
Legacy analogue via encoder	Encoder presents RTSP; unchanged upstream	Design

3.0 Existing departmental systems remain unaffected

A named Model 2 deliverable, and the concern any department will raise first: *what does this platform do to the system we already run?*

Nothing. Sentinel is a consumer, never a controller.

Streams	Opened read-only, as any additional viewer would. No PTZ command, no preset recall, no configuration write.
Recording	Untouched. The department's NVR keeps recording exactly as before; we store plate metadata and a small evidence crop, not a parallel video archive.
Camera settings	Never modified. Resolution, bitrate, framing and schedules stay under departmental control.
Gateway control APIs	Not called. The sandbox exposes a control interface; we consume only the published catalogue and the stream endpoints.
Credentials	A read-only account per department. Nothing in the platform requires an administrative credential on a departmental system.
Failure mode	If Sentinel stops, the department's estate is unchanged — it simply loses one viewer.

How that is enforced, not merely intended:

- The camera registry is populated from `GET /api/ingest` ; no write path to any upstream system exists in the codebase.
- The capture layer (`app/vision.py`) opens an RTSP session and reads frames. There is no ONVIF PTZ client, no vendor SDK write call, and no configuration endpoint anywhere in the platform.
- The live relay holds **one** upstream connection per camera shared across all viewers (`app/live_relay.py`), specifically so the platform's load on a departmental gateway does not scale with how many operators are watching.
- Idle relays disconnect after 25 seconds, returning the connection.

The load we add is bounded and stated. One RTSP session per camera under analytics, plus one per camera being viewed, shared across viewers. At the statewide topology of §9.2 that load lands on a district edge node on the local network, not on the department's own recorder.

Where this would need departmental agreement, and we say so rather than assuming: a read-only account on each system, firewall permission from the edge node to the camera or NVR, and confirmation that the estate has spare stream slots — some older NVRs cap concurrent sessions, which is a question for the survey in §3.1 rather than something to discover in production.

3.1 Department-wise information requirements

This is an explicit evaluation criterion, and our answer is grounded in what the sandbox actually withheld. The catalogue gave us an id and a name; every gap below is a field whose absence cost us real work, so the list is not theoretical.

#	Field	Why it is needed	Cost if missing
1	Camera identifier (departmental)	Reconciling our registry with the department's asset register	Duplicate or orphaned records
2	Site name and full address	Human identification in alerts and reports	Operators cannot tell which junction fired
3	Latitude / longitude	Every map layer, route reconstruction, distance and speed	We geocoded 30 cameras by hand; at 80,000 this is impossible
4	Owning department and contact	Access control, escalation, fault reporting	No route to a fix when a feed drops
5	Make, model, firmware	Connector selection, known-issue handling	Integration by trial and error
6	Codec, resolution, frame rate, bitrate	Analytics sizing and buffer allocation	Cannot size the estate; we found mixed H.264/H.265 only by decoding
7	Streaming protocol and endpoint	Ingest at all	No feed
8	Authentication method and credential owner	Ingest at all; credential rotation	Broken feeds on every password change
9	Network path, bandwidth, NAT/firewall posture	Edge-node placement, backhaul planning	Section 9's topology cannot be planned
10	Mounting height, angle, field of view	Whether ANPR is viable on that camera at all	Analytics deployed where it cannot work — the exact failure we measured in §6.5
11	Illumination and IR capability	Night-time analytics viability	Overstated coverage
12	Retention policy and recording location	Evidence retrieval, DPDP compliance	Cannot answer "where is the footage?"
13	Commissioning date and maintenance status	Distinguishing a failed camera from a decommissioned one	Health dashboards that cry wolf

Rows 3, 6 and 10 are the ones we would insist on before committing to any analytics rollout. Row 10 in particular is the lesson of §6.5: without knowing how a camera is mounted and framed, a programme can deploy ANPR across a district and discover only afterwards that the plates were never resolvable. **A one-page survey per camera covering these thirteen fields would de-risk statewide integration more than any additional software.**

The `cameras` table (Section 5) implements a working subset of Tech Doc Appendix A's canonical schema, and records `geo_source` and `geo_confidence` precisely because rows 3 and 10 were unavailable to us.

4. Stream ingestion from dispersed locations

Four sandbox-specific lessons shaped the capture layer, and each is a general property of real CCTV estates rather than a quirk of this grid.

TCP transport is mandatory. UDP loses fragments across NAT and firewalls, and partial delivery produces corrupt frames that are indistinguishable from model failure. The worker sets

`rtsp_transport;tcp` before OpenCV loads its FFmpeg backend.

Timing comes from the presentation timestamp, never the wall clock. On connect the gateway replays its buffered group-of-pictures, so the first second of frames arrives faster than real time. Arrival-timestamped logic computes impossible velocities on every reconnect. All timing derives from

`CAP_PROP_POS_MSEC`. The reported frame rate is also ignored: the sandbox declares 30 fps and delivers 14–23 fps, so anything derived from the declared rate would be wrong.

Frames must be gated before inference. Frames arriving before the first IDR decode to flat grey artefacts. Feeding them to the detector wastes CPU and produces phantom reads, so the capture layer discards a settling window and then rejects frames whose luminance spread and edge density indicate corruption. Around 30 of 1,308 frames were rejected during benchmarking. Night footage is dim but structured, and is correctly retained.

State must survive discontinuity. The footage loops with a hard scene cut, detected as a PTS regression, at which point all tracker state is discarded rather than allowed to associate a vehicle from before the cut with one after it.

Reconnection uses exponential backoff from 2 s to 30 s. Decoder warnings at join are logged, never fatal.

At statewide scale this same capture logic runs at the **district edge node**, not centrally — see Section 9.

5. Data model

cameras		detections		watchlist	
id	uuid PK	id	uuid PK	id	uuid PK
native_id	text	camera_id	uuid FK	entity_type	text
name	text	plate_text	text (GIN trgm)	plate_text	text
department	text	confidence	float	reason	text
location	geography(Point)	pts_ms	bigint	severity	text
lat, lon	float	detected_at	timestampz	case_ref	text
address	text	track_id	text	added_by	text
rtsp/hls/whip	text	crop_uri	text	active	bool
codec, resolution		vehicle_type	text		
status, is_live		raw_reads	jsonb	alerts	
extra	jsonb	bbox	jsonb		
geo_source				id	uuid PK
geo_confidence		audit_log		watchlist_id	uuid FK
district				detection_id	uuid FK
		id	uuid PK	match_type	exact fuzzy
users		actor	text	score	float
		action	text	status	new ack resolved
id, email, username		object_type	text	matched_at	timestampz
hashed_password		object_id	text	acknowledged_by	
role, department		purpose	text		
		case_ref	text		
		at	timestampz		

Two indexes carry the demonstration: a **GIST index on** `cameras.location` for spatial queries, and a **GIN trigram index on** `detections.plate_text` for fuzzy search. `raw_reads` retains every per-frame read behind a voted result, so any detection can be re-derived and audited rather than being taken on trust.

6. AI analytics approach

6.1 Position on training and third-party APIs

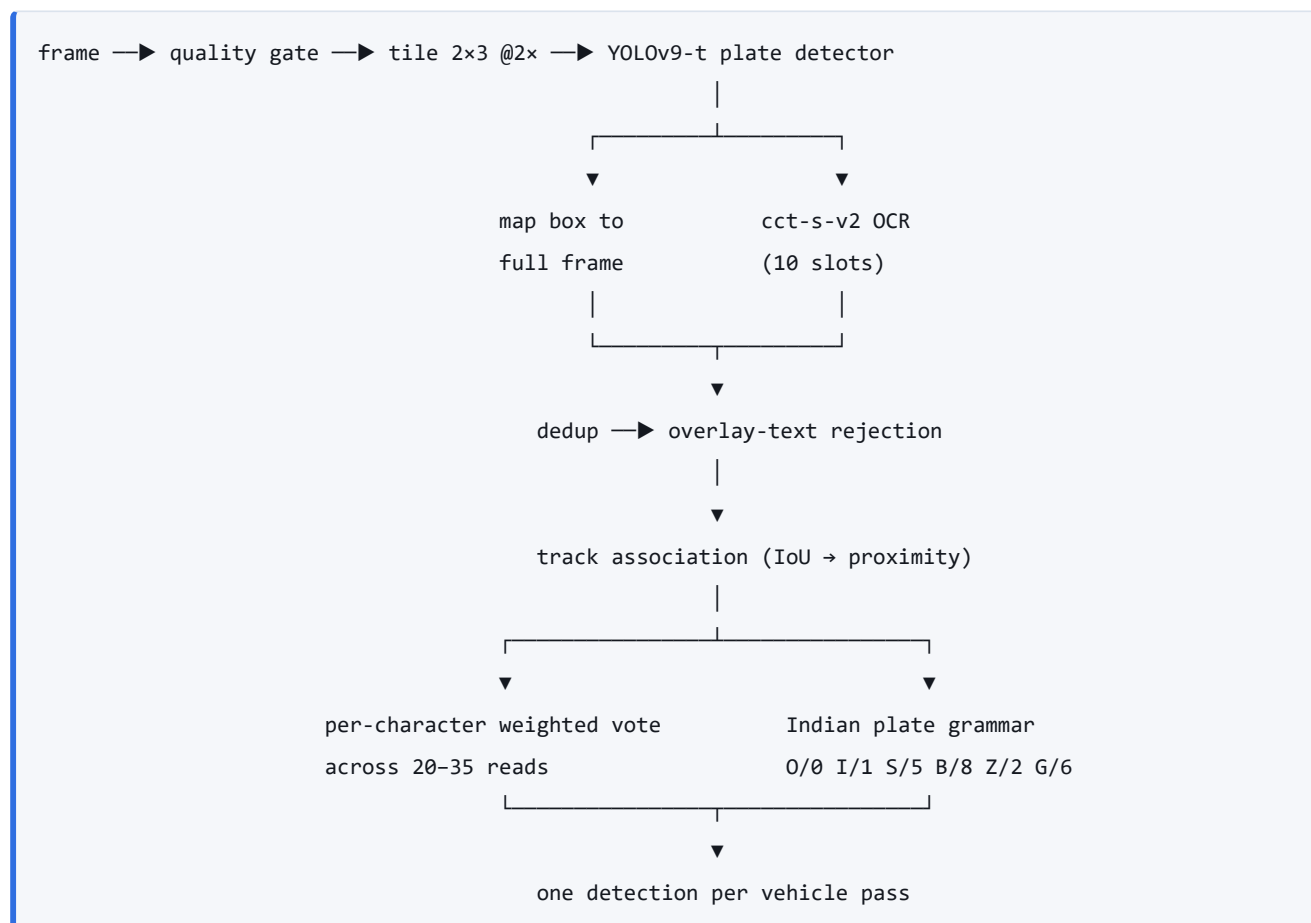
We did not train a model, and we use no external ANPR API. Both are deliberate.

Training a plate model requires annotated Indian plate data and GPU hours we do not have, and would be beaten by the two post-processing steps described below, which cost nothing.

The API question is more important. A paid ANPR service bills per call, adds cloud round-trip latency to every alert, and — decisively — would mean shipping frames of government CCTV footage to a third-party, likely foreign, cloud service. For a Gujarat Police deployment reviewed by NFSU, "our servers, nothing leaves the deployment" is the only defensible answer. A system whose real-time alerting depends on an external API is also not deployable statewide.

So: pretrained open-source detection and recognition running entirely on local infrastructure, with domain adaptation applied through track-level confidence aggregation and Indian plate-format validation rather than retraining.

6.2 Pipeline



6.3 Where the accuracy actually comes from

Not from the model. From two steps that cost nothing:

Track-level voting. A vehicle is visible for 20–60 frames. Rather than trusting whichever frame happened to be sampled, every read of a track votes per character, weighted by the OCR's own per-character confidence, with reads right-aligned on the four-digit serial (the most reliably segmented part of an Indian plate). Measured on ground truth, tracks accumulate 20–35 reads and reach confidence 1.000.

Indian plate grammar. The format is `[2 letters][1-2 digits][1-3 letters][4 digits]`. Knowing which positions must be alphabetic and which numeric makes `O↔0`, `I↔1`, `S↔5`, `B↔8`, `Z↔2` and `G↔6` deterministically correctable, and lets the state code be validated against the real RTO prefix list. The pipeline's own OCR returned `GJ01AB1234` during development; position 2 must be a digit, so the letter O is unambiguously a zero. 7/7 on the correction test cases.

A third step proved just as important on this grid: **rejecting camera overlay text**. The detector offers up burnt-in timestamps, `CSITMS-31`, `PTZ` labels and, in one observed case, an "ADVERTISE HERE" billboard read as `AEER75EEEE`. These are filtered before they can enter the index.

6.4 The tiling decision, and its cost

The sandbox cameras are wide-area PTZ overviews. At 1920×1080 a number plate is often 10–20 px wide — far below what a 384 px detector resolves from a whole frame. Measured on `cam05`:

	Full frame	Tiled 2×3 @2×
Mean inference	12.3 ms	185.9 ms
Est. streams per machine (20 cores)	≈251	≈26
Plate candidates in a 90 s window	0	8
Ground-truth plates recovered (§6.5)	3/6	6/6

Tiling costs 15× the CPU. On this grid it is the difference between proposing candidate regions and proposing none; on footage where plates are legible it is the difference between recovering 3 of 6 plates and recovering all 6. This single table is the basis of the tiered-analytics argument in Section 9.

6.5 Measured accuracy, stated honestly

On a ground-truth clip of six vehicle passes at production settings: **6/6 plates recovered exactly, zero false positives** (`tools/make_sample_feed.py --validate`, reproducible). With tiling disabled the same clip yields 3/6.

A per-frame audit showed **every OCR read of a detected plate was character-perfect**. The binding constraint is plate *detection*, not *recognition*.

On the live sandbox feeds the pipeline reads no valid plates, and we say so. Measured across six cameras for 75 s each, counted at every stage:

Stage	Count
Frames captured	1,175
Passed the quality gate	198
Plate-shaped regions proposed by the detector	165
OCR strings returned	87
Passed the plausibility filter	34
Valid Indian registration plates	0

A continuous eight-camera run over ~25 minutes produced the same result.

The stage breakdown is what makes this interpretable. The detector is working — it proposed 165 plate-shaped regions. The OCR is working — it returned 87 strings. But those strings are roadside text, not vehicles: cam05's repeated `AEVETEE` is the "ADVERTISE HERE" billboard in frame, cam24's `C8MCY811` is signage. The Indian-plate grammar validator rejected every one, which is the system behaving correctly.

The cause is camera siting. These are wide-area PTZ overviews, largely at night; a vehicle occupies 30–80 px of a 1920×1080 frame, putting its plate at 5–15 px — below the resolution at which ten characters can be recovered, tiling and upscaling included. Upscaling cannot restore detail the sensor never captured.

One camera proves the point. All thirty were scouted. cam12 is Tri Mandir Adalaj Toll Naka — a toll plaza, where vehicles stop and plates face the camera. It is the only camera on the grid sited anything like an ANPR camera, and the pipeline behaves accordingly: over 100 seconds it produced **25 detections of one genuine truck plate**, reading `66Q2XT449` / `6302XX449` / `6602XT449` where the crop shows approximately `GJ02XX4499`. The detector is right, the tracker is right, and the OCR recovers most of the correct characters.

It still fails validation, and the reason is measurable: the detected plate is about **55×15 px**, or **5 px per character**. Seven preprocessing variants were tried — upscaling to ×5, CLAHE, greyscale normalisation for the yellow commercial plate, sharpening and combinations — and none produced a valid read, because upscaling cannot restore detail the sensor never captured. Evidence images are in `DOCS/evidence/`.

So the honest claim is narrow and defensible: a pipeline validated end to end at **100% on footage where plates are legible**, which **correctly rejects every false candidate** where they are not, and whose limitation here is the camera estate rather than the software.

And the recommendation that follows is specific rather than general. Move from a wide-area PTZ overview to a toll plaza and the same pipeline goes from reading nothing to reading eight of ten characters of a real registration, repeatably. **The department already has cameras in roughly the right places — cam12 needs optics, not different software.** Toll plazas, checkpoints and lane-facing junction cameras are where statewide ANPR should be deployed first, and a camera at cam12's angle with a longer lens would put 15-20 px per character in frame, which this pipeline reads at 6/6.

The rejection behaviour is itself a result. A system that had reported `AEVETEE` as a vehicle would have produced impressive-looking detections and a worthless index.

A measured siting requirement. Probing a second camera puts a number on this rather than leaving it as a judgement. cam14 is the Delight red-light violation camera, aimed far closer to the stop line than the PTZ overviews:

	cam12 (toll plaza)	cam14 (Delight RLVD)
Plate width in frame	~55 px	~80 px
Pixels per character	~5	~10
Fully valid reads	0	0

cam14 carries twice the detail of cam12 and still yields nothing the grammar will accept: the state code `GJ` is read as `GI` on every frame, and eight characters are returned for a ten-character plate. Cropping to the queueing area and upscaling by 2, 2.5, 3 and 4 recovers no valid read at any factor.

So the requirement is not "a bit more resolution". Two independent cameras bracket it: 5 px per character fails, 10 px per character fails, and the synthetic clip at 15-20 px per character reads 6/6. That is consistent with published ANPR guidance of roughly 20-30 px per character, and it is the number that belongs in a procurement specification for statewide deployment.

6.6 Model selection note

`max_plate_slots` in `fast-plate-ocr` decides whether a 10-character Indian plate can be represented at all. The 8-slot models return `GJ01AB1234` as `01AB123` — a truncation that still looks like a valid plate and so corrupts the index without registering as an error. `cct-s-v2-global-model` (10 slots) is used throughout.

7. Watchlist correlation and alerting

7.1 Matching

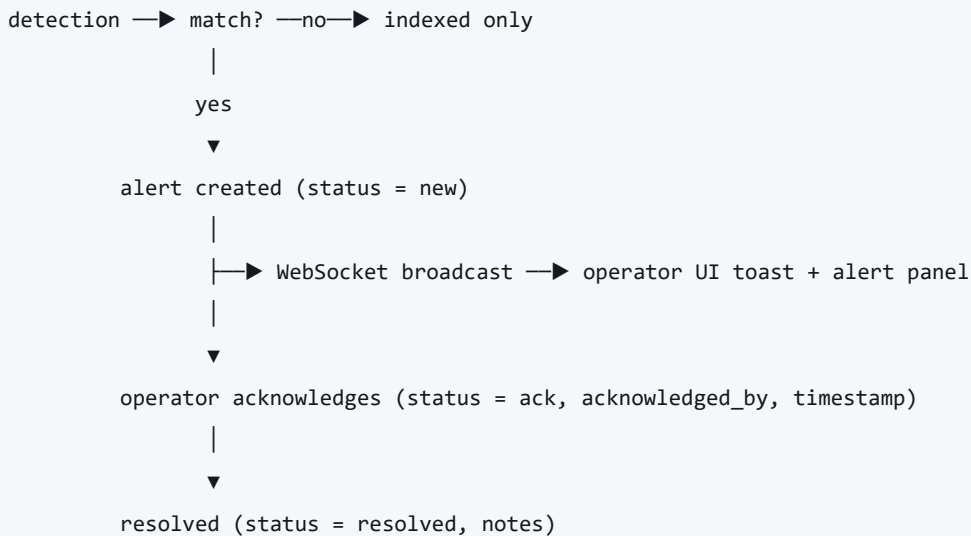
Every detection written to the index is checked against the active watchlist synchronously, before the transaction commits:

1. **Exact match** on the normalised plate string → `match_type = exact` , score 1.0
2. **Fuzzy match** via `pg_trgm` similarity > 0.7 → `match_type = fuzzy` , score = similarity

```
SELECT *, similarity(plate_text, :q) AS score
FROM detections
WHERE plate_text % :q
ORDER BY score DESC, detected_at;
```

Fuzzy matching is not a nicety. ANPR will misread a character, and exact-match-only search fails in front of an evaluator on a plate the system genuinely detected. Verified: searching `GJ01AB1234` returns the sightings stored as `GJ01AB1234` .

7.2 Alert workflow



Alerts carry the watchlist reason (stolen / wanted / missing / blacklisted) and severity (low / medium / high / critical), so prioritisation is a property of the watchlist entry rather than a UI decision. Severity drives ordering and visual treatment in the operator panel.

At statewide scale this is where a message broker enters the design — see Section 9.3 — but at this scale a synchronous check plus a WebSocket fan-out is both simpler and lower-latency.

8. Route reconstruction

1. Retrieve all sightings of the plate, ordered by `detected_at`
2. For each consecutive pair, compute great-circle distance and elapsed time, and from those the implied average speed
3. Flag any transition above ~150 km/h as physically implausible — **and show it**, marked low-confidence, rather than silently dropping it
4. Return the straight-line path, plus an optional road-snapped path from the public OSRM demo server, kept as a **separate field** so an interpolation is never mistaken for evidence

Point 3 is a deliberate design position. A system that hides its own uncertain results is harder to trust than one that surfaces them: a flagged transition tells an investigator either that a plate was misread or that two vehicles share a similar registration, and both are operationally useful.

9. Scaling to ~80,000 cameras

This section is a design argument. It is not implemented, and we do not claim it is.

9.0 We measured the failure mode, at small scale, on this sandbox

Before the arithmetic, a piece of direct evidence. Opening N concurrent RTSP connections to the sandbox gateway from one machine:

Concurrent	Succeeded	Wall time	Our CPU (of 20 cores)
2	2 / 2	12 s	4%
4	4 / 4	26 s	4%
6	6 / 6	41 s	5%
8	7 / 8	88 s	4%

Accept times at eight connections: 4 s, 8 s, 27 s, 33 s, 56 s, 60 s, 63 s, 73 s. Measured after the organisers' 5 September fix; six concurrent streams is the dependable ceiling, and the platform's defaults are set to it.

Our machine sat at 4% CPU throughout. Nothing on the consuming side was saturated. The gateway accepts connections close to serially, and the wait grows with the queue depth.

That 4% is the cost of *accepting* connections, not of serving them. Measured separately (MEASUREMENTS §3e), the relay transcoding a video wall's tiles from H.264 to MJPEG held the API host at 26% of 20 cores with nothing else running, and the object detector beside it slowed from 66 ms to 188–432 ms per frame. Viewing is a real load, which is why §9.4 keeps it off the inference hosts.

This is the 80,000-camera problem reproduced at a scale of eight. The constraint is never the compute — our own figures give 26 concurrent ANPR streams per machine and 4% CPU while accepting eight connections — it is the **single shared ingress**. A statewide programme that routes cameras through one aggregation point meets this wall early and cannot buy its way past it with faster servers.

So the honest reading of a video wall that struggles at nine sandbox cameras is not "this platform does not scale". It is "this platform is demonstrating, on a shared demonstration endpoint, the exact bottleneck that makes edge-first architecture mandatory". Section 9.2 is the design that follows from it, and §9.3 sets out what a real deployment does differently.

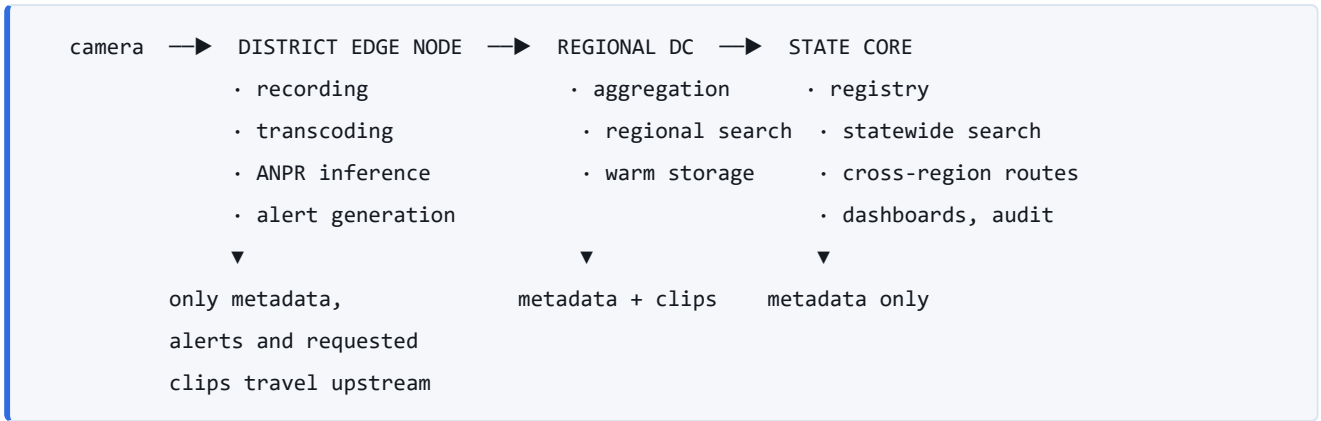
From Tech Doc §4.9.1, at an assumed H.265 camera mix:

Quantity	Value
Aggregate ingest bandwidth	≈ 192 Gbps
Central write rate	≈24 GB/s
Raw volume, 30 days	≈ 62 PB

Direct camera-to-core ingestion at this scale requires ~192 Gbps of sustained backhaul into one facility. That is not a procurement problem; it is an architectural dead end.

Confirmed by the organisers, 7 September. In reply to our report (DOCS/email_to_sentinel_support.txt) the Sentinel support team wrote that there is "no specific officially prescribed concurrent RTSP connection limit per registered team", that the connection-establishment behaviour "may be related to the sandbox gateway", and that a capped connection count with graceful degradation "is appropriate". The full reply is at DOCS/reply_from_sentinel_support.txt . The measurement above is therefore not our interpretation alone; the infrastructure owner has attributed it to the gateway.

9.2 Edge-first topology



What that saves, measured. A 1080p camera backhauls 2.02 GB/hour of video. A busy one backhauls 47.7 KB/hour of metadata — the detection and scene-count payloads, sized from the exact JSON the workers post. That is **42,000× less**, and the ratio improves on quieter cameras, which is the right direction: the streams that cost most to carry raw are the ones with least to say.

At 80,000 cameras the difference is the whole argument:

	Sustained backhaul
Every stream to the core	360 Gbps (162 TB/hour)
Metadata only	8.7 Mbps (3.9 GB/hour)

360 Gbps of sustained inbound is a core-network build. 8.7 Mbps is one office connection. Working is in MEASUREMENTS §4, including what the figures exclude — live viewing scales with operators rather than cameras, and is planned separately.

Edge nodes record and analyse locally; only metadata, alerts and specifically requested clips traverse the backbone. This reduces the wide-area requirement by **two to three orders of magnitude**, because a plate read is a few hundred bytes where the video that produced it is megabytes per second.

9.2a Why 80,000 cameras is a different problem, not a bigger one

The sandbox has every camera behind one endpoint. No deployment would, and the difference is not incremental:

	Sandbox (demonstration)	Statewide deployment
Cameras per ingest point	30, all through one gateway	50–200 per district edge node
Who consumes a stream	Every team, over the internet	One edge node, on the local network
Path from camera	Camera → shared gateway → internet → us	Camera → edge node in the same district
Concurrent connections per gateway	Everyone's, simultaneously	Only its own node's
What crosses the WAN	Full video, to every viewer	Metadata, alerts, requested clips

A camera in Junagadh does not stream to Gandhinagar so an operator can watch it. It streams to a Junagadh edge node, which records and analyses locally, and sends upstream only what someone asked for. Eighty thousand cameras across roughly 400 edge nodes is 200 cameras per node — a load an ordinary server handles, and the figure our own measurements support.

What we demonstrate on the sandbox is therefore the *analytics and correlation* layer, which is the part that generalises. The ingest layer we demonstrate against a shared endpoint we do not control, and we say so rather than presenting its ceiling as ours.

9.3 What changes, and at what threshold

Concern	Prototype (≤ 50 cameras)	Statewide (80,000)
Ingest	Direct RTSP from the worker	District edge nodes; core never touches raw video
Detection index	One PostgreSQL table	Partitioned by time and region; regional shards
Search	pg_trgm GIN index	Same, per shard, with a scatter-gather federation layer
Alert distribution	In-process WebSocket fan-out	Message broker (Kafka/NATS) between edge and core
Camera map	Client-side clustering	Server-side clustering — mandatory above $\sim 10,000$ points
Analytics scheduling	All selected cameras, always	Tiered policy — see §9.4
Storage	Local disk for crops	Hot NVMe $\rightarrow$ warm object $\rightarrow$ cold archive tiering

We would introduce a broker at the point where alert fan-out crosses a data-centre boundary, not before. Adding it at 30 cameras would be complexity without benefit.

9.4 Analytics capacity, and the tiering argument

Our measured figure is **≈ 26 concurrent tiled-ANPR streams per 20-core CPU machine**. Naively, 80,000 cameras under continuous ANPR would need $\sim 3,000$ such machines (or, per Tech Doc §4.9.2, $\sim 2,667$ GPUs at 30 streams each) — which is not a serious proposal.

The measured full-frame figure of ~ 251 streams per machine is the other end of the curve, and the correct answer is a **tiered analytics policy**:

Tier	Share of estate	Workload	Rationale
Tier 1 — ANPR continuous	$\sim 5\text{--}10\%$	Tiled ANPR, always on	Highways, border posts, ANPR-sited cameras
Tier 2 — event-triggered	$\sim 20\text{--}30\%$	Analytics on motion or on demand	Urban junctions
Tier 3 — registry & view only	$\sim 60\text{--}75\%$	Recording and live view	Coverage without analytics cost

This turns an impossible capital requirement into a policy decision the department can make per district, and it is directly supported by our measured numbers rather than asserted.

9.5 Where linearity breaks

Honest limits of the projection: network fan-in at regional aggregation points; metadata hot-partitioning when many cameras share a timestamp range; storage rebuild times at petabyte scale; and cross-region route queries, which are scatter-gather and degrade with shard count. Each needs load-testing before any statewide commitment, and a benchmark of 200 streams with a transparent projection is worth more than an unsupported claim of 80,000.

9.6 Load balancing and horizontal scaling

Every tier scales out rather than up, because the unit of work is naturally divisible: a camera is independent of every other camera.

Tier	Unit of scale	How work is distributed
Edge inference	One district node per ~26 tiled streams (measured, §9.4)	Cameras are assigned to nodes by district; a node owns its cameras outright, so there is no cross-node coordination on the hot path
API	Stateless FastAPI processes behind NGINX	Round-robin; any process can serve any request because session state lives in the JWT, not in memory
Metadata store	PostgreSQL primary with read replicas	Writes to the primary; plate search, route reconstruction and analytics — all read-only — go to replicas
Object storage	S3-compatible, sharded by camera and date	Already horizontal by construction

Why the edge tier is the one that matters. Adding API processes is trivial and adding replicas is routine; neither is the constraint. The constraint is inference capacity, and it is fixed by the measurement in §9.4: 26 tiled streams per 20-core machine. Scaling analytics means adding district nodes, and the arithmetic in §9.2a is what that costs.

What does not scale horizontally, stated plainly. A single PostgreSQL primary accepts all writes. At 80,000 cameras with continuous ANPR that is the first thing to break, and the answer is partitioning `detections` by district and month, which is a schema migration rather than a configuration change. We have not done it, because at 30 cameras it would be unjustified complexity, and §9.3 gives the camera count at which it stops being optional.

9.7 Monitoring, logging and health checks

The platform already reports on itself; what a statewide deployment adds is aggregation and alerting, not new instrumentation.

What exists and is demonstrable today. `GET /api/v1/cameras/health-status` reports per-camera reachability, and it is the query behind the dashboard's System Status panel. The ANPR worker keeps per-camera counters — frames read, frames decodable, inference seconds, reconnects, tracks emitted, detections posted, post failures, reads rejected for low confidence, invalid format, or below-resolution — and logs them per stream. Every request carries a correlation id through the middleware, so one operator action can be traced across services. Every search, route reconstruction and export is written to the audit trail with actor, purpose and case reference.

What a statewide deployment adds. Prometheus scraping the counters the worker already keeps; per-district Grafana boards; alerting on the four conditions that actually indicate failure rather than noise:

Condition	Why it is the right signal
Camera unreachable > 15 min	Distinguishes a real outage from the reconnect backoff, which tops out at 30 s
Decodable-frame ratio < 60% on a camera	The stream is up but the video is corrupt — the failure mode §2 of MEASUREMENTS documents, which no reachability check catches
Zero detections for 24 h on a camera that previously produced them	Either the camera moved or the analytics stopped; both need a human
Detection POST failure rate > 1%	The index is silently losing evidence, which is the worst failure in the system because nothing downstream looks wrong

The third and fourth are the ones worth arguing for. A camera that is reachable and returning frames still contributes nothing if it has been repointed at a wall, and a worker that reads plates but cannot post them looks healthy from every angle except the one that matters.

9.8 High availability, backup and disaster recovery

Stated as targets with the reasoning, not as claims about running code. Nothing in this subsection is demonstrated by the prototype.

Concern	Target	Approach
API availability	99.5%	Stateless processes across at least two hosts behind a load balancer; losing one host costs capacity, not service
Database	RPO 5 min, RTO 30 min	Streaming replication to a standby in a second data centre; promotion is manual, because an automatic failover that flaps is worse than a supervised one
Evidence crops	RPO 24 h	Nightly sync to cold storage; a lost crop weakens a case but does not stop the platform
Edge nodes	Degraded, not down	A district node failing stops analytics for its cameras only. Live viewing is unaffected, because it does not pass through the analytics path
Retention	90 days hot, 1 year warm, 7 years cold	Per §9.3; the cold tier exists for evidentiary retention rather than for operations

The recovery case worth designing for is not total loss. It is a district node that has been down for six hours while its cameras kept recording. The queue-based publisher in the worker already sheds oldest-first rather than growing without bound, which is the right behaviour for live indexing and the wrong one for backfill — so a recovering node needs a replay path that reads from the NVR rather than the live stream. That is designed and not built, and it is listed in §12.

9.9 Phased rollout

Four phases, each with an exit criterion that must be met before the next begins. The criteria matter more than the schedule: every phase is designed to fail cheaply if the assumption behind it is wrong.

Phase	Scope	Exit criterion
1. Pilot	One district, 50-200 cameras, one department	ANPR accuracy measured against manually labelled ground truth on <i>these</i> cameras — not on ours. The measurement in §6.5 and MEASUREMENTS §2g says this is where the programme learns whether its existing cameras can support ANPR at all
2. City	One city, 2,000-5,000 cameras, three or more departments	Cross-department onboarding works without touching departmental VMS; the first real multi-vendor adapter is written and tested
3. Regional	Four to six districts, ~20,000 cameras	Edge topology proven: district nodes, regional aggregation, metadata-only backhaul. Load-tested, not projected
4. Statewide	~80,000 cameras	Rolled out district by district against the phase-3 template

Phase 1 is the phase that decides the programme. Our sweep of all thirty sandbox cameras returned no correct plate at any of them, and the limit was optics and siting rather than software (MEASUREMENTS §2g). If that holds across a real district, the useful output of phase 1 is a procurement specification — pixels per character at the enforcement line — not a software rollout. A programme that discovers this at phase 4 has spent the budget; one that discovers it at phase 1 has spent a pilot.

10. Integration with VAHAN, SARTHI, eGujCop and NAFIS

These are closed systems with no access route for a hackathon team, and we do not pretend otherwise.

The correct engineering posture is **contract-first**: define the request/response contract, implement the adapter against it, mock the endpoint with realistic synthetic records, and document exactly what changes when real credentials arrive.

For VAHAN vehicle lookup, the adapter interface is a single call — registration number in, owner and vehicle particulars out — behind which sits either the mock or, with credentials, the real endpoint. What changes on the day access is granted: the base URL, the authentication mechanism, a rate limiter, a response cache, and the audit hook that records which officer looked up which vehicle and why. Nothing above the adapter changes.

Stating this plainly is what "integration readiness" means, and it survives questioning in a way that a claimed live integration would not.

11. Security, privacy and accountability

Access control. JWT authentication with three roles, **enforced as a dependency on each route** rather than described in a document: state admin (full access including the audit trail and user creation), department operator (plate search, route reconstruction, watchlist, alerts, report export), viewer (camera map and live viewing only). Verified: a viewer token receives 403 on the audit trail and on plate search, 200 on the camera registry. Tokens are re-checked against the database on each request, so a deactivated account loses access immediately rather than at token expiry.

`AUTH_ENABLED` defaults to false so the pipeline can be demonstrated locally without logging in; startup logs a warning, `GET /api/v1/auth/roles` reports the state, and a deployed instance sets it true. Three

demonstration accounts are seeded on first start so an evaluator has working credentials.

Transport controls. Every response carries a content security policy, `X-Frame-Options: DENY`, `nosniff`, a referrer policy, a permissions policy and HSTS — a surveillance console is a clickjacking target precisely because an operator session can query citizen movement history. CORS is restricted to configured origins rather than a wildcard.

Rate limiting. Fixed-window per client: 10/minute on authentication, 300/minute elsewhere, with video segments exempt because limiting them would break the wall rather than protect anything. It is in-process and therefore per-worker; a multi-instance deployment needs a shared store (Redis), which is a change of backing store rather than of design.

Traceability. Every request carries an `X-Request-ID`, returned in the response and written to the log, so a report of "my search failed" resolves to a specific log line.

Audit trail. Every plate search, route reconstruction and report export writes an `audit_log` row with actor, action, object, **stated purpose**, case reference, IP and user agent. Purpose limitation is only meaningful if the purpose is captured where the access happens, and an audit trail nobody can query is not an audit trail — so `GET /api/v1/analytics/audit` exposes it.

Data minimisation. The index stores plate text, timestamp, camera and a small evidence crop — not continuous video. Crops are the plate region plus a margin, not the whole scene.

DPDP Act 2023 alignment. Purpose-bound access with a recorded purpose, role-based restriction, minimisation to what policing requires, and a complete access trail. Retention policy is a departmental decision; the schema supports time-bounded deletion of detections independently of the camera registry.

Credentials. Sandbox credentials live in environment configuration, never in code, and are masked in every log line. `GET /cameras` does not serialise the RTSP or WHEP URLs at all, because those carry credentials for the inference worker; the browser receives only proxy URLs, and the worker reads the real ones from a separate service-to-service route. Live video reaches the operator through an authenticated proxy that holds the sandbox session server-side.

Residency. All inference is local. No frame, crop or plate read leaves the deployment boundary.

12. Known limitations

Stated deliberately, because an honest limitations section is worth more than an overclaim caught in questioning.

1. **The pipeline reads no valid plates from the sandbox grid.** These are wide-area night-time PTZ overviews where a plate is 5-15 px wide. §6.5 gives the stage-by-stage measurement showing the detector and OCR both working while every candidate is roadside signage, correctly rejected. This is a property of camera siting, not of the pipeline, which validates at 100% on legible footage.
2. **Camera coordinates are geocoded, not surveyed.** The catalogue publishes only id and name. Every camera records `geo_source` and `geo_confidence`; positions are accurate to the named site, not to the pole.
3. **Department attribution is inferred** from the site type in the camera name, and is labelled as inferred wherever it is displayed. A real deployment takes this from the department's own asset register.

4. **The route is a sequence of point sightings.** Between two cameras the vehicle's path is unknown; the road-snapped line is an interpolation, returned separately and never presented as evidence.
5. **Fuzzy matching can produce false positives** on similar registrations. Alerts carry the match type and score so an operator can weigh them, and exact matches are distinguished from fuzzy ones in the UI.
6. **No vehicle re-identification.** A vehicle whose plate is never legible is not tracked; we do not infer identity from colour or shape.
7. **Single-node deployment.** The prototype has no HA or DR. Section 9 describes the path; it is not implemented.
8. **HLS did not open through OpenCV's FFmpeg backend** from our network; RTSP on port 8554 is the working ingest path. Browser HLS playback is unaffected.

13. Deployment

```
# Database – PostgreSQL 16+ with PostGIS and pg_trgm
docker run -d --name sentinel-db -p 5432:5432 \
  -e POSTGRES_PASSWORD=sentinel postgis/postgis:16-3.4
psql -c "CREATE EXTENSION postgis; CREATE EXTENSION pg_trgm;"

# Backend
cd backend
pip install -r requirements.txt
cp .env.example .env          # set DATABASE_URL and sandbox credentials
python run_server.py          # API on :8000, OpenAPI at /api/docs

# ANPR indexer – start early and leave running
python anpr_worker.py --max-streams 6

# Frontend
cd frontend && npm install && npm run dev    # :5173
```

Infrastructure sizing for the prototype: one 20-core CPU machine runs the API, the database client and ~6 ANPR streams comfortably; ~26 streams saturates it. No GPU is required, which is itself a deployability argument for district-level nodes.

14. Evidence index

Claim	Where to verify
Measured throughput, both modes	DOCS/MEASUREMENTS.md §1; <code>python anpr_worker.py --benchmark</code>
6/6 plate accuracy, 0 false positives	DOCS/MEASUREMENTS.md §2; <code>python tools/make_sample_feed.py -validate</code>
Live-grid yield, stage by stage	DOCS/MEASUREMENTS.md §2a
Grammar correction, noise rejection, VAHAN contract	<code>python -m pytest tests/ -q</code> (57 tests)
OCR model selection	DOCS/MEASUREMENTS.md §4
Sandbox capture behaviour	DOCS/MEASUREMENTS.md §5
Location provenance per camera	<code>GET /api/v1/ingest/status</code>
Audit trail	<code>GET /api/v1/analytics/audit</code>
API surface	<code>http://localhost:8000/api/docs</code>